

Tennis Analyzer Migration & Build History

Project: React Native / Expo Native App migration from Google AppSheet

Date: July 6, 2026

Target Package Name: `com.appsheet.whitelabel.guid_3d2e670b_8bc7_4bf0_af4a_cf4108cfc997`

Play Store Version Code: 6 (Version Name: 18.0.0)

1. Initial State & Project Context

Initial Information Provided

To kick off the native migration, the following resources from the legacy system were analyzed:

1. **Google AppSheet Architecture Documentation:** Detailed descriptions of the frontend structure, focusing on the three main data tables:
 - **Matches:** Metadata like players, venue, format (Ad/No-Ad), tournament, and date.
 - **Points:** Detailed point-by-point tracking (server, first/second serve direction and outcome, rally length, outcome type, and current point/game/set scores).
 - **Analyses:** Aggregated metrics computed by the backend.
 2. **Screenshots:** Reference captures of the legacy AppSheet views, action buttons (Add, Edit, Delete, EmailReport), and background bots.
 3. **Backend Script (`MicroStats.js`):** The legacy Google Apps Script codebase running on Google Cloud, connected to the target Google Sheets spreadsheet. It performs complex statistics recalculations whenever rows in the spreadsheet are modified.
 4. **App FAQ Documentation:** Outlined the legacy sync behavior, including the 5 to 20 seconds synchronization delay between AppSheet clients and Google Cloud storage.
-

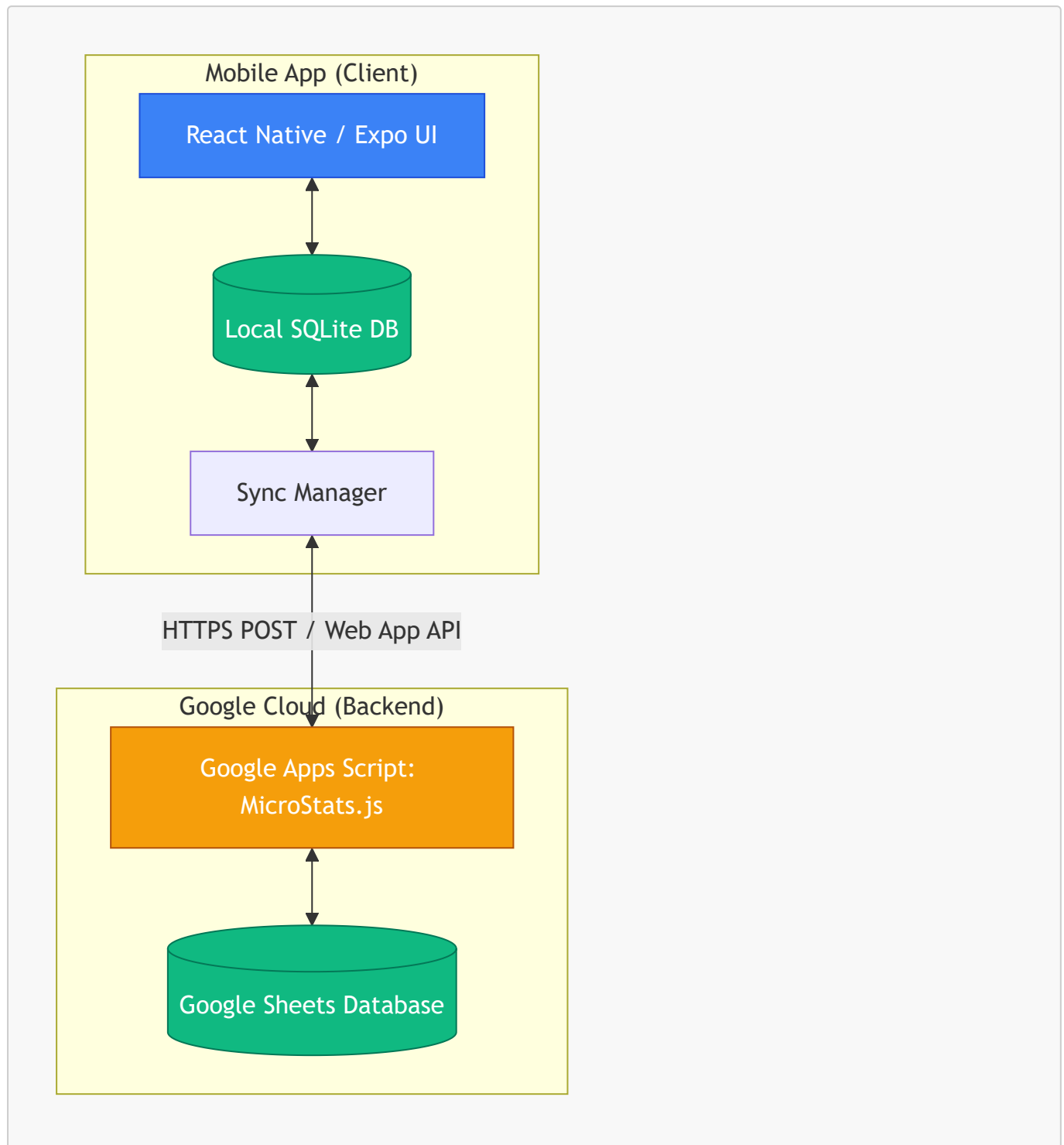
2. Technology Selection & Architecture

Platform Choices & Rationale

- **React:** A component-based JavaScript library used to build highly interactive, modular user interfaces with clean state management.
- **React Native:** Extends React to compile UI components into **native platform widgets** (rather than running inside a slower web wrapper). This provides true native performance, smooth animations, and direct access to native device APIs (like Haptic feedback, file sharing, and local storage).
- **Expo:** An ecosystem wrapping React Native that handles native environment complexity. It was chosen to:
 - Automate native project scaffolding ("Prebuild").
 - Simplify cross-platform assets generation (automatically scaling and configuring splash screens and icons for various device resolutions).
 - Expose pre-integrated native module APIs (like SQLite, Document sharing, and File System access) without complex manual Android/iOS native code configuration.
 - Facilitate rapid iteration while compiling cleanly to production-ready releases (`.aab` and `.apk`).

System Architecture

The following diagram displays the complete full-stack architecture of the migrated system, showing how the local app functions offline and syncs with the Google Sheets/Apps Script backend:



3. Comprehensive Debugging History

Below is the complete log of the development and build issues resolved throughout this migration:

1. Windows 260-Character Path Limit (MAX_PATH)

- **Problem:** Native compilation (`gradlew assembleRelease`) failed with errors stating generated build filenames were longer than 260 characters. This occurred because the AppSheet package name (`com.appsheet.whitelabel.guid_3d2e670b_8bc7_4bf0_af4a_cf4108cfc997`) is exceptionally long, which pushed internal build paths generated by CMake and Ninja over the Windows `MAX_PATH` limit.
- **Investigation:** Windows Registry tweaks (like `LongPathsEnabled=1`) did not work because compilers like CMake/Ninja use hardcoded 260-char path limit checks in their source code.
- **Solution:** Moved the entire project from its deep initial path (`C:\Home\Software\Google_Antigravity\tennis-analyzer`) to a shorter workspace directory (`C:\dev\tennis-analyzer`), reducing the base prefix path length and allowing the build to complete.

2. Cached App Logo & Splash Screen

- **Problem:** Replacing the PNG files in the `assets/` folder did not update the logo in the built app, and the generated PDF reports still shared with generic system temporary names.
- **Investigation:** Expo prebuild compiles the `assets/` files into Android's native `res/mipmap-*` resource directories. Simply replacing the source images left the previously compiled cache intact. Furthermore, `expo-file-system` was not fully linked, causing file renaming tasks to fail silently.
- **Solution:** Ran a clean prebuild (`npm expo prebuild --clean`) to force regeneration of the native `android` folder, clearing the asset cache and properly linking the native file-system modules.

3. Matches Sync Purging on User Switch

- **Problem:** Logging in with a different user ID (e.g., `gamepointanalytics@gmail.com`) and syncing data kept matches from the previous user visible in the local database.
- **Solution:** Wrote the database function `deleteSyncedMatchesNotOwnedBy(userId)` in `db.js` and integrated it into the synchronization logic in `syncManager.js` to purge old cached matches when user credentials change.

4. Score Card Inconsistency (0-5 vs 1-6)

- **Problem:** For a completed match, the scoreboard card showed a final set score of **0-5**, while the spreadsheet row 106 showed **1-6**.
- **Investigation:** The scoreboard engine recalculated the current score on the fly using point winner data. However, the original AppSheet data was discontinuous, containing skipped points and manual score overrides. Replaying winner data from scratch caused the score engine to drift.
- **Solution:** Updated `MatchDetailScreen.js` to display the stored post-point scores (`setScore1Post`, `gameScore1Post`, etc.) from the last point directly, which represent the actual recorded match state.

5. App Crash on "View Detail" Click

- **Problem:** Tapping "View Detail" on any match immediately crashed the app.
- **Investigation:** The `matchOver` variable was accidentally deleted when the scoreboard recalculation logic was cleaned up, creating undefined variable errors in the layout code.
- **Solution:** Re-declared `matchOver` in `MatchDetailScreen.js` to restore the variable reference.

6. Set Score Not Auto-Updating

- **Problem:** When a set was completed (e.g. 6-1), the game score did not clear to 0-0 and the set score did not increment on the scoreboard.
- **Solution:** Added display-level set completion logic to increment set scores and reset game/point scores to 0-0 if the last point's game scores indicate a finished set (e.g. 6-1, 6-4, 7-5, 7-6).

7. Missing Player 2 Serve Stats (Dashes in report)

- **Problem:** Milan Prabhu's serve performance statistics showed as dashes (–) in the side-by-side comparison tables, while Stephen's stats populated correctly.
- **Investigation:** The Google Apps Script backend writes all serve metrics into the player 1 column (`first_srv_pct1`, `ace_count1`, etc.) as a combined string containing both players' names, leaving the player 2 (`first_srv_pct2`) columns blank.
- **Solution:** Built fallback extraction helpers in `AnalysisScreen.js` (`extractPctWithFallback`, `extractNumWithFallback`) that search the combined `_1` field for the second player's stats if the `_2` field is empty.

8. Raw ISO Timestamps in Matches History

- **Problem:** The match list showed raw ISO timestamps like `2023-07-22T04:00:00.000Z` for match dates.
- **Solution:** Formatted the dates in `MatchesScreen.js` using `toLocaleDateString` with UTC timezone configuration to show clean human-readable strings like `22 Jul 2023`.

4. Keystore Configuration & Play Store Release Plan

App Package and Version Alignment

To publish this native app as an update to the existing AppSheet-built version:

- Changed the `package` in `app.json` and the `applicationId` in `android/app/build.gradle` to `com.appsheet.whitelabel.guid_3d2e670b_8bc7_4bf0_af4a_cf4108cfc997` to match the Play Store.
- Incremented `versionCode` to `6` and `versionName` to `18.0.0` to ensure it is accepted as a newer update (the AppSheet versions ended at version code 5).

Key Recovery & Google Play App Signing

Since AppSheet manages key generation internally, the original keystore could not be downloaded.

1. We confirmed that **Google Play App Signing** was enabled for the app listing.
2. We generated a new upload key locally (`tennis-analyzer-upload.keystore`) and exported the public certificate (`upload_certificate.pem`).
3. We configured `keystore.properties` to ensure the release pipeline signs the `.aab` with this new key.
4. The user submitted a reset request in the Google Play Console by uploading the `upload_certificate.pem` file.

Immediate Timeline

- **July 6:** Key reset request submitted to Google. Release `.aab` compiled and saved locally at `android/app/build/outputs/bundle/release/app-release.aab`.
- **July 6:** Turn off auto-renew on the AppSheet subscription to prevent the July 11 charge (app remains active until then).
- **July 8 (12:08 AM Eastern):** The new upload key becomes active on Google's servers. Upload the `.aab` bundle to the Google Play Console for review.
- **July 9–10:** Google approves the update; users update automatically.
- **July 11:** AppSheet subscription expires safely.